C PROGRAMMING — COMPLETE TEACHER'S NOTES

Topic: File Handling in C

Prepared for teaching secondary-school beginners. These notes follow the uploaded 5-page document in order and explain the visible material, programmes, tables, outputs and Exercise 4.9 Questions 1–7.

PAGE 1 — INTRODUCTION TO FILE HANDLING

1. What is File Handling?

Simple meaning: File handling means using a C programme to create, open, read, write to, and close files on a computer.

Think about a school notebook. You can open it, write information in it, read information from it, and close it. A computer file works in a similar way.

```
students.txt

John
Mary
Peter
Asha
```

A file allows information to be kept for later use.

Why Do We Need Files?

A normal variable stores information while a programme is running. A file allows information to remain available after the programme stops.

2. Permanent Storage

Permanent storage means information can remain available even after the programme stops. A useful classroom analogy is: a variable is like writing on a board, while a file is like writing in a notebook that can be opened again later.

3. Handling Large Data

Files are useful when there is a large amount of information, such as records for hundreds or thousands of students. It would be difficult to keep all of that information in separate variables.

4. Sharing Data

One programme can write information to a file and another programme can open the same file and use the information.

Easy memory: Files help us keep data, store large data, and share data.

Visible programme before File Handling

```
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

The array contains five numbers. The **for** loop visits positions 0 to 4 and printf() displays each element.

Output: 1 2 3 4 5

Question 4 — Full Name Input

The document gives a programme intended to take a full name as input. The issue is that scanf("%s", name) normally reads one word and stops at whitespace.

```
char name[20];

printf("Enter your name: ");
scanf("%s", name);

printf("Hello, %s!\n", name);
```

For example, entering **John Peter** will normally read only **John**. For a complete line, a beginner-friendly alternative is fgets().

```
char name[50];

printf("Enter your full name: ");
fgets(name, sizeof(name), stdin);

printf("Hello, %s", name);
```

Question 5 — Count Characters Without strlen()

Use a loop to count characters until the C string terminator **'\0'** is reached.

```
#include <stdio.h>

int main() {
    char word[100];
    int count = 0;

    printf("Enter a word: ");
    scanf("%s", word);

    while (word[count] != '\0') {
        count++;
    }

    printf("Number of characters = %d\n", count);
    return 0;
}
```

Question 6 — puts() and printf()

printf() displays formatted output and can use format specifiers such as %d, %f, %c and %s.

```
int age = 18;
printf("Age = %d\n", age);
```

puts() is mainly used to display a string and moves to the next line.

```
puts("Hello students");
```

Function	Main use
printf()	Prints formatted output
puts()	Prints a string

Start of File Handling

The document introduces file handling as a way of storing and retrieving data from files on a computer. Instead of using printf() and scanf() only with the console, file-handling functions allow programmes to read from and write to files.

PAGE 2 — READING AND WRITING FILES IN C

C provides file-handling functions for opening, reading, writing and closing files.

Table 4.7 — Basic File-Handling Functions

Function	Purpose
fopen()	Opens a file
fclose()	Closes a file
fprintf()	Writes data to a file
fscanf()	Reads data from a file

1. fopen()

fopen() means open a file. It is used before reading or writing.

```
FILE *file;
file = fopen("myfile.txt", "w");
```

FILE is the C type used for file streams. FILE *file creates a file pointer variable. For beginners, think of it as the variable that lets the programme work with the opened file.

2. fclose()

fclose() closes a file after we finish using it.

```
fclose(file);
```

Easy rule: **Open** → **Work** → **Close**.

3. fprintf()

fprintf() writes formatted data to a file.

```
fprintf(file, "Hello students!");
```

4. fscanf()

fscanf() reads formatted data from a file. It is similar to scanf(), but the source is a file.

Important comparison

Normal input/output	File input/output
scanf() — keyboard/input	fscanf() — file
printf() — screen	fprintf() — file

Opening a File — fopen()

```
FILE *file;
file = fopen("myfile.txt", "w");
```

The filename is **myfile.txt**. The mode **"w"** tells C that we want to write.

Table 4.8 — File Modes

Mode	Meaning
"r"	Read mode; file must exist
"w"	Write mode; creates a new file or overwrites an existing file
"a"	Append mode; adds to the end of the file
"r+"	Read and write mode

"w+"	Write and read mode
"a+"	Append and read mode

"r" — READ MODE

Use it to read an existing file. The file should already exist.

```
file = fopen("data.txt", "r");
```

"w" — WRITE MODE

Use it to write. It can create a new file, and if the file already exists, its previous contents can be overwritten.

```
file = fopen("data.txt", "w");
```

"a" — APPEND MODE

Append means add new information at the end. Existing information is preserved.

```
file = fopen("data.txt", "a");
```

"r+" — READ AND WRITE

Allows reading and writing; the file needs to exist.

```
file = fopen("data.txt", "r+");
```

"w+" — WRITE AND READ

Allows writing and reading; it can create the file and can overwrite an existing file.

```
file = fopen("data.txt", "w+");
```

"a+" — APPEND AND READ

Allows reading and appending; new writing is added at the end.

```
file = fopen("data.txt", "a+");
```

Easy memory: r = read, w = write, a = add at the end.

PAGE 3 — WORKING WITH TEXT FILES

Working with text files in C includes creating, reading, writing and closing files using the standard I/O library **stdio.h**. Text files store data as plain characters.

Writing to a File Using fprintf()

fprintf() is a standard C I/O function that writes formatted text to a file, much like printf() writes to the console.

Programme Example 4.19

```
#include <stdio.h>

int main() {
    FILE *file = fopen("myfile.txt", "w");

    if (file == NULL) {
        printf("Error in opening a file!\n");
        return 1;
    }

    fprintf(file, "Hello, this is a file handling example!\n");
    fprintf(file, "File handling makes data storage easy.\n");

    fclose(file);

    printf("Data written to file successfully.\n");
    return 0;
}
```

Line-by-line teaching explanation

• #include gives access to standard I/O and file functions.

• int main() is where the programme starts.

• FILE *file creates the file pointer.

• fopen("myfile.txt", "w") opens/creates myfile.txt in write mode.

• if (file == NULL) checks whether opening failed.

• return 1 stops the programme when there is an opening error.

• fprintf(file, ...) writes the first line into the file.

• The second fprintf() writes the second line.

• fclose(file) closes the file after use.

• printf(...) displays a confirmation message on the console.

• return 0 ends the programme successfully.

Content written into myfile.txt:

```
Hello, this is a file handling example!
File handling makes data storage easy.
```

Console output shown in the document:

```
Data written to file successfully.
```

Important: fprintf() writes to the file, while the final printf() writes to the screen.

Teacher's simple explanation

Students, imagine a file as a notebook. We open it, write information inside it, close it, and later we can open it again to read the information.

PAGE 4 — READING FROM A FILE USING fscanf()

fscanf() reads formatted data from a file. It is similar to scanf(), which normally reads input from the keyboard.

Programme Example 4.20

```
#include <stdio.h>

int main() {
    FILE *file = fopen("myfile.txt", "r");

    if (file == NULL) {
        printf("Error in opening a file!\n");
        return 1;
    }

    char text[100];
    fscanf(file, "%s", text);

    printf("File content: %s\n", text);

    fclose(file);
    return 0;
}
```

The programme opens myfile.txt in read mode, creates a character array called text, reads one word using fscanf(), prints it on the screen, and closes the file.

Output shown in the document:

```
File content: Hello
```

Important warning about %s

With fscanf(file, "%s", text), %s normally reads one word and stops at whitespace. If the file contains 'Hello students', it will normally read 'Hello'. To read a complete line, a function such as fgets() is more appropriate.

Exercise 4.9 — Question 1

The document gives a programme that opens data.txt in read mode and then tries to write to it using fprintf().

```
FILE *file;
file = fopen("data.txt", "r");
fprintf(file, "Hello, file handling!");
```

Problem: "r" is read mode, but fprintf() is trying to write.

Fix: Use an appropriate writing mode, such as "w".

```
file = fopen("data.txt", "w");
```

A safer complete version also checks file == NULL before writing.

Exercise 4.9 — Question 2

Create info.txt and write the two required lines.

```
#include <stdio.h>

int main() {
    FILE *file = fopen("info.txt", "w");

    if (file == NULL) {
        printf("Error opening file.\n");
        return 1;
    }

    fprintf(file, "Learning C is fun!\n");
    fprintf(file, "File handling makes programming easier.\n");

    fclose(file);
    return 0;
```

```
}
```

The \n at the end of each string moves the next text to a new line.

Exercise 4.9 — Question 3

Read and print the contents of message.txt. A simple solution using fscanf() reads one word.

```
#include <stdio.h>

int main() {
    FILE *file;
    char text[100];

    file = fopen("message.txt", "r");

    if (file == NULL) {
        printf("Error opening file.\n");
        return 1;
    }

    fscanf(file, "%s", text);
    printf("File content: %s\n", text);

    fclose(file);
    return 0;
}
```

For a complete sentence or multiple lines, fgets() with suitable processing is more appropriate.

PAGE 5 — EXERCISE 4.9 CONTINUED

Question 4

The document asks for a programme that reads one word from data.txt. The shown code uses the correct read mode and fscanf() pattern. The important improvement is to check whether fopen() succeeded before reading.

```
#include <stdio.h>

int main() {
    FILE *file;
    char word[50];

    file = fopen("data.txt", "r");

    if (file == NULL) {
        printf("Error opening file.\n");
        return 1;
    }

    fscanf(file, "%49s", word);
    printf("Read: %s\n", word);

    fclose(file);
    return 0;
}
```

%49s limits the number of characters read so there is room for the terminating '\0' in word[50].

Question 5

The document asks us to stop overwriting notes.txt and instead add new text to the end.

Problem: The original programme uses "w". Write mode can overwrite existing contents.

Fix: Change "w" to "a".

```
#include <stdio.h>

int main() {
    FILE *file;

    file = fopen("notes.txt", "a");

    if (file == NULL) {
        printf("Error opening file.\n");
        return 1;
    }

    fprintf(file, "C programming is powerful.\n");

    fclose(file);
    return 0;
}
```

If notes.txt already contains 'C is easy.', after running it may contain both the old line and the new line.

Question 6

Count and print the number of characters in story.txt. A character-by-character approach uses fgetc(), because the exercise asks for characters rather than one word.

```
#include <stdio.h>

int main() {
    FILE *file;
    int count = 0;
    char ch;

    file = fopen("story.txt", "r");

    if (file == NULL) {
```

```
        printf("Error opening file.\n");
        return 1;
    }

    while ((ch = fgetc(file)) != EOF) {
        count++;
    }

    printf("Number of characters = %d\n", count);

    fclose(file);
    return 0;
}
```

fgetc() reads one character at a time. The loop continues until EOF (end of file). Each character increases count by one.

Question 7

The shown programme opens notes.txt for reading, reads one word, prints it, and then returns. Two important issues to teach are: it does not check whether fopen() returned NULL, and it does not close the file.

```
#include <stdio.h>

int main() {
    FILE *file;
    char text[100];

    file = fopen("notes.txt", "r");

    if (file == NULL) {
        printf("Error opening file.\n");
        return 1;
    }

    fscanf(file, "%99s", text);
    printf("Read: %s\n", text);

    fclose(file);
    return 0;
}
```

COMPLETE FILE-HANDLING PROCESS

Students can remember the complete process as:

```
OPEN
  ↓
CHECK
  ↓
READ / WRITE
  ↓
CLOSE
```

• Include .

• Declare FILE *file.

• Open the file with fopen().

• Check whether file == NULL.

• Read with fscanf() or write with fprintf(), where appropriate.

• Close with fclose().

printf() VS fprintf()

printf()	fprintf()
Displays output on the screen	Writes formatted output to a file

printf("Hello");	fprintf(file, "Hello");

scanf() VS fscanf()

scanf()	fscanf()
Reads normal input, commonly from keyboard	Reads formatted data from a file
scanf("%s", word);	fscanf(file, "%s", word);

Common Beginner Mistakes

• Forgetting to include when using these standard I/O functions.

• Using "r" when trying to write.

• Using "w" when old data must be preserved; use "a" to append.

• Forgetting fclose(file).

• Not checking file == NULL after fopen().

• Confusing printf() with fprintf().

• Confusing scanf() with fscanf().

• Assuming %s reads a complete sentence containing spaces.

Student Check Questions

Files: What is a file? Why do we use files? What is permanent storage? Why are files useful for large data?

Functions: What does fopen() do? What does fclose() do? What does fprintf() do? What does fscanf() do?

Modes: What does "r" mean? What does "w" mean? What does "a" mean? Which mode adds information at the end?

Key Points to Remember

```
FILE *file;  → file pointer
fopen()      → open
fprintf()    → write
fscanf()     → read
fclose()     → close

r  → read
w  → write
a  → append
```

COMPLETE REVISION OF THE WHOLE SECTION

• File handling lets a C programme work with stored data in files.

• Files provide permanent storage, support large data, and can help share data between programmes.

• The main functions in the document are fopen(), fclose(), fprintf(), and fscanf().

• The main modes are r, w, a, r+, w+, and a+.

• printf() is used for screen output; fprintf() is used for formatted file output.

• scanf() normally gets input from the keyboard; fscanf() reads formatted data from a file.

• A good basic sequence is OPEN → CHECK → READ/WRITE → CLOSE.

• Always consider what the chosen file mode will do to existing data.

What Students Must Know for an Exam

• Define file handling in simple words.

• State reasons for using files.

• Explain fopen(), fclose(), fprintf(), and fscanf().

• Explain the six file modes.

• Distinguish printf() from fprintf() and scanf() from fscanf().

• Write a simple programme to create/write to a file.

• Write a simple programme to read from a file.

• Identify and fix incorrect file modes.

• Explain why append mode is used when existing content must be preserved.

• Recognise the need to check fopen() and close the file after use.

Teacher's Final Classroom Explanation

Students, today we are learning File Handling in C. A file is a place where our computer can store information. We use files because information stored in them can remain available even after our programme stops.

Before we work with a file, we open it using fopen(). We tell C the file name and how we want to use it. For example, "r" means read, "w" means write, and "a" means add information at the end.

After opening the file, we can write information using fprintf() or read information using fscanf(). When we finish, we close the file using fclose().

The easiest way to remember file handling is to think about a notebook: first open it, then read or write in it, and finally close it.

Source Coverage Check

The notes cover the uploaded 5-page document in sequence: Page 1, Page 2, Page 3, Page 4 and Page 5, including the visible material before File Handling, the File Handling introduction, Tables 4.7 and 4.8, Programme Examples 4.19 and 4.20, their outputs, and Exercise 4.9 Questions 1–7.

Source: uploaded CamScanner PDF, 5 pages.